Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Output Format

The output displays whether the system is in a safe state or not. If the system is in a safe state, the program prints the Safe Sequence of processes. If the system is not in a safe state, the program indicates that a deadlock may occur.

Example 1

Sample Input 1

3

3

0 1 0

2 0 0

3 0 3

3 2 2

2 1 1

4 3 3

0 0 0

Sample Input 1

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is NOT in a safe state (Deadlock may occur).

Example 2

Sample Input 2

3

2

1 0

0 1

1 1

2 1

1 2

3 3

1 1

Sample Input 2

Enter number of processes:

Enter number of resource types:

Enter Allocation Matrix:

Enter Max Matrix:

Enter Available Resources:

System is in a SAFE STATE.
Safe Sequence: P0 P1 P2

CODE:

```
#include <stdio.h>

int main()
{ int n, r, i, j, k;
   printf("Enter number of processes: ");
   scanf("%d", &n); printf("Enter number of
   resource types: "); scanf("%d", &r);
   int alloc[n][r], max[n][r], need[n][r], avail[r];
   int finish[n], safe_sequence[n]; int count = 0,
   safeFound;

   // Allocation Matrix printf("Enter
   Allocation Matrix:\n"); for(i = 0; i <
   n; i++)
   { for(j = 0; j < r; j++)
      { scanf("%d", &alloc[i][j]);
      }
   }

   // Max Matrix printf("Enter
   Max Matrix:\n"); for(i = 0; i <
   n; i++)
   { for(j = 0; j < r; j++)
      { scanf("%d", &max[i][j]);
      }
   }

   // Available Resources printf("Enter
   Available Resources:\n");
   for(i = 0; i < r; i++)
   { scanf("%d", &avail[i]);
   }

   // Initialize finish array for(i
   = 0; i < n; i++)
   { finish[i] = 0;
   }
```

```
// Calculate Need Matrix for(i
= 0; i < n; i++)
{ for(j = 0; j < r; j++)
    { need[i][j] = max[i][j] - alloc[i][j];
    }
}

// Banker's Algorithm while(count
< n)
{ safeFound = 0;

    for(i = 0; i < n; i++)
    { if(finish[i] == 0)
        { int safe = 1;

            for(j = 0; j < r; j++)
            { if(need[i][j] > avail[j])
                { safe = 0;
                    break;
                }
            }

            if(safe)
            { for(k = 0; k < r; k++)
                { avail[k] += alloc[i][k];
                }

                safe_sequence[count]    =
                i; finish[i] = 1; safeFound
                = 1;
                count++;
            }
        }
    }

    if(!safeFound)
    { printf("System is in unsafe state! May occur deadlock\n");
        return 0;
    }
}

printf("Safe sequence is: "); for(i
= 0; i < n; i++)
{ printf("P%d ", safe_sequence[i]);
}
```

```
    return 0;
}
```

OUTPUT:

Custom Test Success

Input :

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```